Operator Overloading (Operator Functions)

- Operator functions
  - Redefine operators (+,−,*,/,[],<<,>>,…) as functions
  - Member function method
    - Use **`this`** keyword to implicitly get argument
    - Gets left operand for binary operators (like **+**)
    - Leftmost object must be of same class as operator
  - Non-member function method
    - Need parameters for both operands
    - Can have object of different class than operator
    - Must be a **`friend`** to access **`private`** or **`protected`** data

C++ : Operator overloading example (non-member function using friend)

```
// operator_overloading.cpp
#include <iostream>
using namespace std;

class Complex {
private:
   double re, im;

public :
   Complex( double r, double i ) : re(r), im(i) {}
   friend Complex operator+(Complex&, Complex&);
   friend Complex operator-(Complex&, Complex&);
   friend Complex operator*(Complex&, Complex&);
   void Display( ) {
               if (im>0) cout << re << "+" << im <<"i" << endl;
               else if (im<0) cout << re << im <<"i" << endl;
               else if (im==0) cout << re << endl;
   }
};

// Operator overloaded using a non-member function
Complex operator+( Complex& i , Complex& other ) {
   return Complex( i.re + other.re, i.im + other.im );
}

Complex operator-( Complex& i , Complex& other ) {
   return Complex( i.re - other.re, i.im - other.im );
}

Complex operator*( Complex& i , Complex& other ) {
   return Complex( i.re * other.re - i.im * other.im
                   , i.re*other.im + i.im*other.re );
}
```

```
int main() {
   Complex a = Complex( 1.2, 3.4 );
   Complex b = Complex( 5.6, 7.8 );
   Complex c = Complex( 0.0, 0.0 );

   a.Display();
   b.Display();
        cout << endl;

   c = a + b; // c = operator+(a,b);
   c.Display();
   c = a - b; // c = operator-(a,b);
   c.Display();
   c = a * b; // c = operator*(a,b);
   c.Display();
}
```

결과:

```
1.2+3.4i
5.6+7.8i

6.8+11.2i
-4.4-4.4i
-19.8+28.4i
```

C++ : Operator overloading example (member function)

```
// operator_overloading.cpp
#include <iostream>
using namespace std;

class Complex {
private:
   double re, im;

public :
   Complex( double r, double i ) : re(r), im(i) {}
   Complex operator+( Complex &other );
   Complex operator-( Complex &other );
   Complex operator*( Complex &other );
   void Display( ) {
                if (im>0) cout << re << "+" << im <<"i"
<< endl;
                else if (im<0) cout << re << im <<"i" << endl;
                else if (im==0) cout << re << endl;
   }
};

// Operator overloaded using a member function
Complex Complex::operator+( Complex &other ) {
   return Complex( re + other.re, im + other.im );
}

Complex Complex::operator-( Complex &other ) {
   return Complex( re - other.re, im - other.im );
}

Complex Complex::operator*( Complex &other ) {
   return Complex( re * other.re - im * other.im
                       , re*other.im + im*other.re );
}
```

```
int main() {
   Complex a = Complex( 1.2, 3.4 );
   Complex b = Complex( 5.6, 7.8 );
   Complex c = Complex( 0.0, 0.0 );

   a.Display();
   b.Display();
        cout << endl;

   c = a + b; // c=a.operator+(b);
   c.Display();
   c = a - b; // c=a.operator-(b);
   c.Display();
   c = a * b; // c=a.operator*(b);
   c.Display();
}
```

```
결과:
1.2+3.4i
5.6+7.8i

6.8+11.2i
-4.4-4.4i
-19.8+28.4i
```